INT161 · Basic Backend Development

Express Framework

Express replaces the manual HTTP code: routing, middleware, and the Controller, Service and Repository layers.

Raw http vs Express | Reading the Request | Handling Responses

Controller, Service, Repository | Routing | Middleware | Project Template | Glossary

01

Raw http vs Express

Express is a small **framework** on top of the http **module**. It keeps the same job, and removes the hand-written parts.

Reading the body

http: Collect stream chunks by hand

Express: `express.json()`

Matching the URL

http: if / else on request.url

Express: `app.put('/api/subjects/:id', ...)`

Reading query and path

http: new URL(...) and split

Express: `req.query and req.params`

Sending the answer

http: writeHead + JSON.stringify + end

Express: `res.status().json()`

Last class: reading the body by hand

```
function getBody(request) {
  return new Promise((resolve, reject) => {
    let body = '';
    request.on('data', chunk => { body += chunk.toString(); });
    request.on('end', () => resolve(body ? JSON.parse(body) : null));
    request.on('error', reject);
  });
}
```

app.js: the same job in Express

```
import express from 'express';
const app = express();
app.use(express.json()); // middleware
app.put('/api/subjects/:id', (req, res) => {
  const { filter, page, size } = req.query;
  const id = req.params.id;
  const subject = req.body;
  subject.id = id;
  res.json([subject, { filter, page, size }]);
});
app.listen(3000);
```

- A framework is a tool with a frame. You write inside the frame, so you must learn how it works.
- Express is **unopinionated**. It does not force a database or a folder layout on you.
- Its core is **routing** and **middleware**. The rest of the class builds on those two.

Postman

A **browser** can only send GET from the address bar. **Postman** lets you pick PUT, POST or DELETE. Use it to test your **API**, because the **front end** is usually not finished yet.

02

Reading the Request

A **request** carries data in three places. Express gives each one its own property.

Path | Query | Body

```
PUT /api/subjects/5?page=2&size=10
{ "title": "Web Technology", "credit": 1 }
```

Read it with

`req.params.id`

You get

`"5"`

The :id in the **route** pattern. It names which subject.

03

Handling Responses

res.json() sends a JavaScript value as **JSON**. res.status() picks the **status code** first.

Sending a response

```
res.json(subject);                  // 200 by default
res.status(201).json(subject);      // set the code, then send
res.status(404).json({ message: 'Subject not found' });
res.status(404);                    // sets the code only, nothing is sent yet
```

> ⚠ **Watch out**
>
> res.status() alone does not send anything. It returns res so you can chain .json() or .send() onto it.

04

Controller, Service, Repository

The lecturer compared the **backend** to a hospital. The request is the patient. Pick a layer.

Controller | Service | Repository

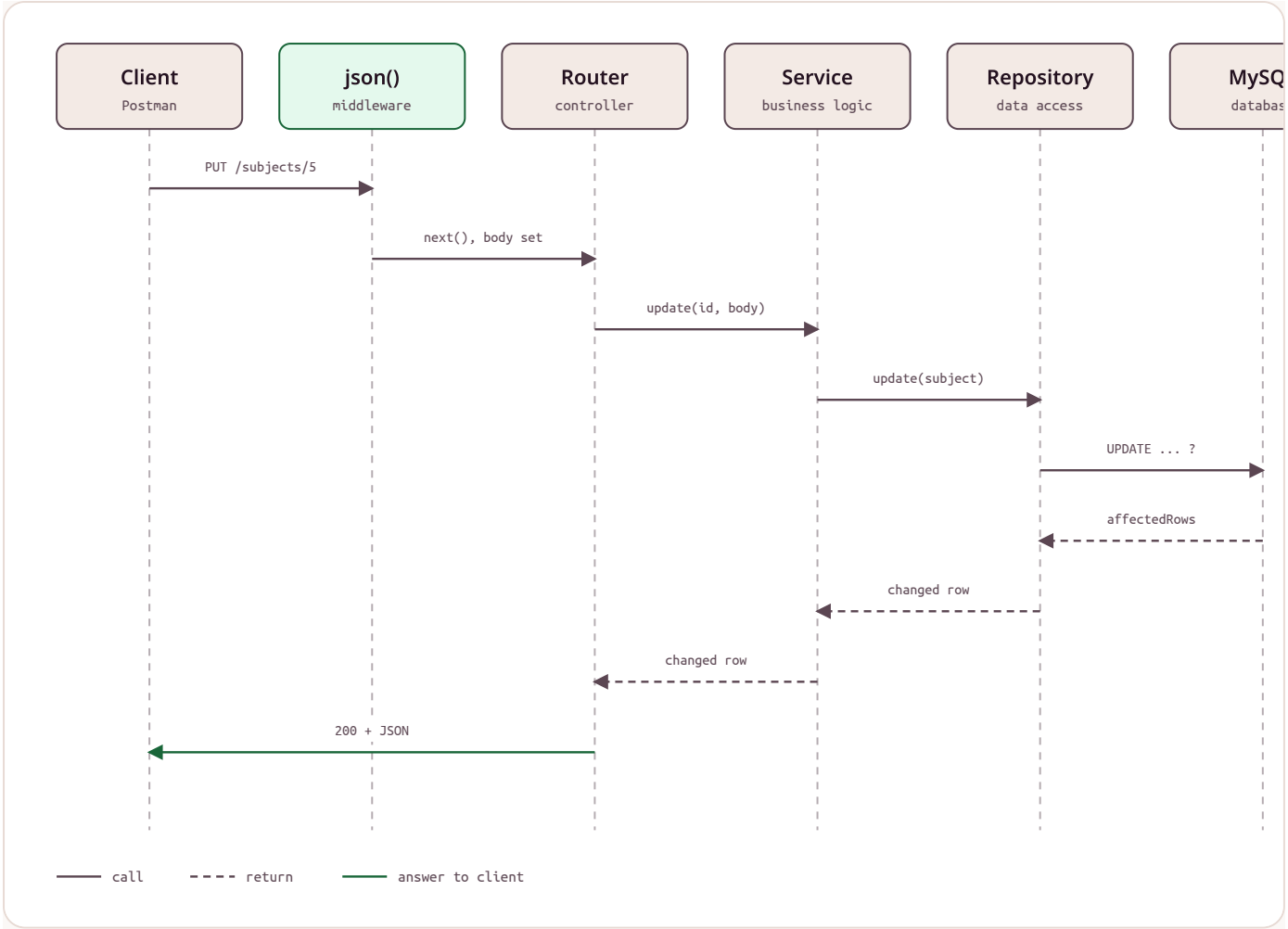
`Client` → `Controller` → `Service` → `Repository` → `Database`

Controller = Reception desk

Takes the **HTTP request**, hands it on, sends the HTTP **response**.

Should contain	Should not contain
Routing, mapping, validation	Business logic, DB code

```
res.status(404).json({ message: 'Subject not found' })
```



- Why three? So code is not repeated. A student **service** can call the subject service instead of rewriting it.
- Each layer talks only to the one below it. Easier to scale and to change.

05

Routing

A **route** is METHOD, **PATH**, HANDLER. Group the routes of one **resource** in a **Router**, then mount it under a prefix.

routes/subject-route.js

```
import express from 'express';
import * as service from '../services/subject-service.js';
const router = express.Router();
router.get('/', async (req, res, next) => {
  const subjects = await service.getAllSubjects();
  res.json(subjects);
});
router.get('/:id', async (req, res, next) => {
  const { id } = req.params;
  const [subject] = await service.getSubjectById(id);
  if (!subject[0]) return res.status(404).json({ message: 'Subject not found' });
  res.json(subject[0]);
});
export default router;
```

app.js: mounting routers

```
import express from 'express';
const PORT = 3000;
const app = express();
app.use(express.json());
app.use('/customers', customerRouter);
app.use('/orders', orderRouter);
app.use('/products', productRouter);
app.listen(PORT, () => {
  console.log('Running at http://localhost:' + PORT);
});
```

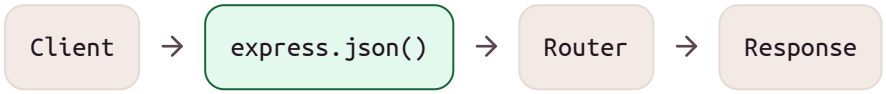
The handler reads the request and calls the service. It does not hold **business logic**.

06

Middleware

Middleware is a function that sits between the request and the router. Order matters. Try the three setups.

json, then router | router, then json | no json



Inside the route handler

```
req.body = { title: "Web Technology"
}
```

Result

```
200 subject saved
```

A **request** passes through the **middleware** in the order you wrote them. The **body** is parsed before the **router** reads it.

A logger middleware

```
app.use((req, res, next) => {
  console.log(req.method + " " + req.url);
  next(); // pass it on, or the request hangs
});
```

- Call **next()** to continue. Send a **response** to stop the chain.
- Typical jobs: parse JSON, check login, log, handle errors, allow CORS.

Application-level | Router-level | Error-handling | Built-in | Third-party

07

Project Template

The course writes **ESM**. Set it up once, then save the project as a template.

package.json

```
{
  "name": "basic-rest-express",
  "version": "1.0.0",
  "main": "src/app.js",
  "type": "module"
}
```

1. Create the project and run **npm** install express.
2. Add "type": "module" to package.json.
3. Create src with configs, repositories, services and routes.
4. Write app.js, a router, a service and a **repository**.
5. In WebStorm: Tools, Save project as template, name it Express-ESM.

08

Glossary

API
: Application Programming Interface: the doors a program offers to other programs.

backend
: The part on the server: it handles data, rules and the database, and answers the front end with JSON.

body
: The data a request carries after the headers. For an API it is usually JSON.

browser
: The client most people use, such as Chrome, Edge or Safari.

business logic
: The rules of the application itself: checks, calculations, and what may happen when.

ESM
: The modern module system: import and export. Needs "type": "module" in package.json.

Express

A small web framework for Node.js. It adds routing and middleware on top of the built-in http module.

framework

A toolkit that makes a job easier but sets the frame you must write inside.

front end

The part users see and touch. It runs in the browser or on the phone.

HTTP

HyperText Transfer Protocol: the plain-text language a browser and a web server use to talk. One side sends a request, the other sends a response.

JSON

JavaScript Object Notation: a plain text format for data that both humans and programs can read.

middleware

A function (req, res, next) that runs between the request and the route handler. It can change the request, answer it, or pass it on.

module

A file of code you load into another file with import.

next()

Call it to hand the request to the next middleware in the chain.

npm

Node Package Manager: installs libraries such as express into node_modules.

PATH

A Windows setting that lists the folders searched when you type a command. If the MySQL bin folder is not in it, mysql is not recognized.

Postman

A tool for sending any HTTP method to your API, so you can test it before the front end exists.

repository

The layer that reads and writes the database. No business rules, no HTTP.

request

What the client sends: method, URL, headers and sometimes a body.

resource

A thing the API gives access to, such as a product or a customer. In this course it maps to a database table.

response

What the server sends back: status code, headers and a body.

route

Matching an HTTP method and a URL path to the function that handles it.

Router

Matching an HTTP method and a URL path to the function that handles it.

routing

Matching an HTTP method and a URL path to the function that handles it.

service

The layer that holds the business rules. It calls repositories and knows nothing about HTTP.

status code

The number that tells the client how it went: 200 OK, 201 Created, 404 Not Found, 500 Server Error.

unopinionated

It does not force a structure. You choose the folders, the database and the layers.